Ders 3 Genişletilmiş Notu: Singleton, Adapter, Bridge, Composite ve Decorator Kalıpları

Ders 3 Genişletilmiş Notu: Singleton, Adapter, Bridge, Composite ve Decorator Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Singleton Kalıbı
- 2. Yapısal Kalıplara Geçiş ve Adapter Kalıbı
- 3. Bridge Kalıbı
- 4. Bridge Örneği: İki Çizim Kitaplığı ile Şekiller
- 5. Composite Kalıbı
- 6. Composite Örneği: Makine ve Makine Parçaları Kataloğu
- 7. Decorator Kalıbı
- 8. Decorator Örneği: Makbuza Üst Bilgi ve Alt Bilgi Eklemek
- 9. Örnek Sorular

1. Singleton Kalıbı

Ders, yaratımsal (creational) kalıpların sonuncusu olan Singleton kalıbıyla açıldı. Kalıbın amacı, bir sınıftan yalnız ve yalnız tek bir nesne oluşabilmesini sağlamaktır: o sınıftan bir nesne olacak, ama ikincisi olamayacaktır. Hocanın örnekleri işletim sisteminden geldi. Bir işletim sisteminde tek bir yazdırma biriktiricisi (print spooler) vardır; bütün yazıcılara gönderilen belgeler onda toplanır ve görevleri o dağıtır. Benzer şekilde tek bir dosya sistemi olur, ikinci ve üçüncü kopyaların oluşturulması istenmez.

1.1 Sorumluluk kime verilmeli?

İlk akla gelen çözüm static bir nesne tanımlamaktır. Fakat bu tek nesne farklı sınıflardaki nesneler tarafından da kullanılmak isteniyor. O zaman nesne bunlardan hangisinin üyesi olacak? Hoca burada nesneye dayalı tasarımın önemli bir sorusunu hatırlattı: bir veri ya da nesne en çok nerede gerekiyorsa oraya verilir. Onu oluşturmak için gereken bilgi birden fazla nesneye yayılmışsa, nesnenin durum bilgisinin en büyük kısmı hangisindeyse ona verilir.

Ne var ki bu sorunun cevabı her zaman net değildir. Üstelik tek nesneyi barındıracak nesne daha ayağa kalkmadan başka bir nesne ona ihtiyaç duyabilir. Bu yüzden en tartışmasız çözüm, tek nesneyi oluşturma sorumluluğunu sınıfın kendisine vermektir. Sınıfın kalıbı da budur.

1.2 Kalıbın yapısı

Slayttaki önerilen çözüm üç maddeyle özetleniyordu (ekrandan, 00:06:00): sınıfın kendi türünden static bir üyesi olur; kurucusu private tanımlanarak kurucuya erişim engellenir; bu üye için sınıfta bir fabrika metodu bulunur. Kalıba adını veren bu sınıfın adı Singleton'dır. UML sınıf şeması şöyledir:

```
Singleton
-----------------------------------------
- uniqueInstance : Singleton {static}
- state : SingletonData
-----------------------------------------
- Singleton( ... )
+ getInstance( ) : Singleton {static}
+ singletonOperation( )
+ getterMethods( )
```

`uniqueInstance` üye alanı ortak kullanım söz konusu olabilsin diye static olmak zorundadır. Singleton nesnesinin durum bilgisini oluşturan üye alanlar da aynı sınıftadır. Şemada bu durum ayrı bir sınıftan (`SingletonData`) bir nesne olarak çizilmiş, ama böyle olması zorunlu değildir; ilkel ya da ilkel olmayan başka üye alanlar da birlikte durum bilgisini oluşturabilir.

Kurucu private tanımlanır, çünkü tek olacak nesnenin herkes tarafından istenildiği gibi oluşturulması istenmez. Böylece kurucuyu yalnız sınıfın kendisi kullanabilir. Hoca, Singleton dışında private kurucu tanımlamaya ihtiyaç duyulan pek pratik bir örnek olmadığını ekledi.

Nesneye `getInstance()` adlı bir fabrika metoduyla (factory method) ulaşılır. Static üyeyi döndüreceği için kendisi de static'tir. `singletonOperation()` ve getter, setter gibi diğer metotlar sınıfın asıl işlerini yapar. Davranışı kritik olan tek metot `getInstance()`'tır. Şemadaki not kutusunda gövdesi şöyle verilmiştir (ekrandan, 00:06:00):

```
if( uniqueInstance == null ) {
    uniqueInstance = new Singleton( ... );
}
return uniqueInstance;
```

Önce `uniqueInstance` alanının henüz ilklendirilmemiş olabileceğine bakılır. Null ise private kurucu `getInstance()` içinden çağrılarak nesne oluşturulur. Zaten ilklendirilmişse bu bloğa girilmez ve metodun sonunda mevcut nesne döndürülür.

Hoca bu kalıp için ayrıca kod örneği yapmadı. Kalıp basittir; UML şeması ve not kutusu onu doğru ve eksiksiz kodlamak için gereken bilgiyi zaten verir. İleride State ve Strategy kalıplarında, oradaki gerçeklemelerin birer Singleton olmaya aday olduğu görülecek.

1.3 Singleton ve kalıtım

Singleton sınıfından kalıtımla yeni, daha özelleşmiş Singleton alt sınıfları türetilebilir. Fakat bu durumda her alt sınıfın nesnesi yalnız kendi içinde tek olur. "Bir alt sınıftan bir örnek varsa diğer alt sınıftan örnek olmasın" gibi bir kural bu kalıpla kodlanamaz. Kalıbın sağladığı, bir Singleton sınıfından tek bir nesne oluşmasıdır.

2. Yapısal Kalıplara Geçiş ve Adapter Kalıbı

Singleton ile yaratımsal kalıplar bitti. Sıradaki grup, Dörtlü Çete'nin (Gang of Four, GoF) kitabında yapısal (structural) kalıplar diye geçen gruptur (ekrandan, 00:10:00: "Kısım 1: Tasarım Kalıpları, 1.2. Yapısal (Structural) Kalıplar").

2.1 Amaç ve priz benzetmesi

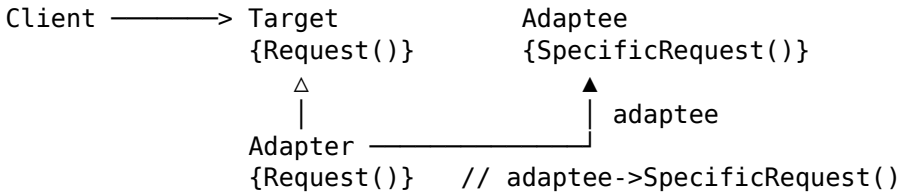
Adapter kalıbının amacı şudur: işlevleri bakımından uyumlu, fakat metot imzaları bakımından uyumsuz bir istekçi (client) ve bir sunucu (server) varsa, bunları birbiriyle konuşturmak.

Hoca bunu yurt dışına çıkanların bildiği bir örnekle anlattı. Ülkelerin elektrik prizi standartları farklıdır. Bazı ülkelerde fiş iki yuvarlak bacaklıdır. İngiliz etkisindeki ülkelerde ise prizde üç delik vardır: altta yan yana iki delik, üstte ortada bir delik. Fişin üstteki üçüncü bacağı biraz daha uzundur ve önce o girer. Girerken prizdeki bir dili aşağı iter, böylece normalde kapalı olan alttaki iki deliğin plastik kapağı açılır. Bu, kazalara karşı daha emniyetli bir prizdir. Ama elinizde Avrupa standardında bir fiş varsa, onu İngiliz prizine takmak için önce üstteki deliği bir şeyle bastırmanız gerekir. Bunun yerine çeşitli elektrik standartlarını birbirine uyduran seyahat adaptörleri kullanılır. Hoca internette "travel adapter" aratıp örnekler gösterdi: bazıları

tek parçadır, bazıları Lego parçaları gibi birbirine takılan birkaç parçadan oluşur. Kalıp da bu mantıkla çalışır.

2.2 Kalıbın yapısı

Kalıbın sınıf şeması şöyledir (ekrandan, 00:22:30):



İstekçi bir iş yaptırmak için `Request()` metodunu çağırmak istiyor. Uyarlanması gereken sınıfta (`Adaptee`) böyle bir metot yok; onun yerine `SpecificRequest()` var. Yaptığı iş tamamen aynı, belki parametre listesi de aynı tür ve sayıda parametre içeriyor, ama adı farklı. Kaynak koduna da erişilemiyor. O zaman araya bir adaptör konur. Adaptör, istekçinin çağırmak istediği metoda sahiptir; bu metot da kendi içinde uyarlanan sınıfın ilgili metodunu çağırır, yani çağrıyı yönlendirir.

Üstteki `Target` arayüzü isteğe bağlıdır. İstekçi doğrudan adaptörle, adaptör de doğrudan uyarlanan sunucu nesneyle konuşabilir. Fakat istekçi farklı türden uyarlanacak sınıflarla konuşacaksa farklı adaptörler gerekir; onları ortak bir arayüz altında toplamak hem yararlı hem gerekli olur. Özetle istekçi ve uyarlanan sınıf zaten mevcuttur. Biz adaptörü ve gerekiyorsa `Target` arayüzünü ya da soyut sınıfını yazarız.

Kalıp, elimizdeki bir sınıfın istekçi tarafından kullanılmasını engelleyen tek şey istekçinin yolladığı mesajla sınıfın anladığı mesajın adlarının (ve belki parametre sırasının) farklı olması olduğunda kullanılır.

2.3 Örnek: çizim programına eski bir daire sınıfını uydurmak

Bir çizim programı üzerinde çalışıyoruz ve programın bir kısmı hâlihazırda kodlanmış. İki boyutlu farklı şekillerimiz var ama ortak işlemleri de var: bir x-y konumunun set ve get edilmesi, şeklin ekrana çizilmesi, içinin doldurulması, renginin değiştirilmesi ve ekrandan silinmesi. Bunlar soyut `Shape` sınıfında toplanmış.

Çizme, doldurma ve renk değiştirme her geometrik şekil için farklı kodlanmak zorundadır; bir doğrunun rengini değiştirmekle bir dikdörtgenin ya da dairenin kenarlarını boyamak farklı geometrik işlemler gerektirir. Bu yüzden bunlar soyut metotlardır. Ekrandan silmek ya da konumu set etmek ise hepsi için aynıdır; bunlar somut metotlardır. Somut şekil sınıfları (nokta, çizgi, kare) kalıtımla türetilmiştir.

Slayttaki şemada `display`, `fill` ve `setColor` metotları italik yazılmıştı. Hoca bunların soyut olduğunun italikten anlaşıldığını, ama şemayı değiştirmeye üşendiği için bunu `{abstract}` notuyla göstermenin daha doğru olacağını söyledi.

Sonra sıra daire sınıfına gelir. Daire çizmek kolay değildir. Ekran kare bir piksel matrisidir; içinden yuvarlak bir şekil geçirmek zordur. Hoca bunu Excel'de hücreleri piksel gibi boyayarak gösterdi (ekrandan, 00:20:00): kare hücrelerle dikdörtgen çizmek kolaydır, ama daireye benzer bir şekil için bazı ara pikselleri daha açık renkle boyamak gerekir. Buna kenar yumuşatma (anti-aliasing) denir ve karmaşık algoritmaları vardır.

Neyse ki zamanında bir daire sınıfı kodlamışız: adı `XXCircle` olsun, ihtiyaç duyulan bütün yetenekleri de olsun. Fakat metot imzaları şu anki altyapıyla uyuşmuyor. Konum set ve get edilebiliyor, ama çizim için bizde `display`, onda `displayIt` var; bizde `fill`, onda `fillIt` var. Parametre sırası da farklı olabilir: daireyi merkezin x-y koordinatları ve yarıçapla tanımlıyoruz, eski kodda ise belki önce yarıçap veriliyor. İşin kötüsü kaynak kodunu kaybetmişiz, elimizde yalnız class dosyaları var ve sıfırdan yazacak vaktimiz yok.

Çözüm Adapter'dır. Soyut `Shape` sınıfından somut `Circle` sınıfı türetilir, ama bu sınıf bir adaptör olarak tasarlanır:

- `Circle`, `XXCircle` türünden bir üye alana sahip olmak zorundadır. Kalıp da adaptörün, uyarlayacağı türden bir nesneye sahip olmasını gösterir.
- `Circle` kurucusu bu üye alanı ilklendirir.
- Örneğin `display()` metodunun yaptığı tek iş, `XXCircle` nesnesinin `displayIt()` metodunu çağırmaktır. Diğer metotlar da kendilerine gelen çağrıyı uyarlanan nesnenin anlayacağı biçime çevirir.

Hoca, şemada bu ilişkinin parça-bütün olarak çizildiğini, ama düz bir sahiplik çizgisiyle de gösterilebileceğini belirtti. Singleton ve Adapter görece basit fikirlerdir; ama günlük hayatta da kullandığımız pratik fikirlerdir.

3. Bridge Kalıbı

3.1 Amaç

Sıradaki kalıp, önceki haftalardakiler gibi daha ayrıntılı bir kalıptır: Bridge (köprü). Amacı, bir nesnenin arayüzünü gerçeklemesinden ayırmaktır. Bu iddialı bir cümle gibi gelir, çünkü nesne tanımı gereği üye alanları ve üye metotlarıyla ayrılmaz bir bütündür. Metotların arayüzünü gövdelerinden nasıl ayırabiliriz? Hoca bunun garip bir şey olmadığını göstereceğini söyledi. Kalıbın bir adı da Handle'dır. Hocaya göre Windows'un ilk, 32 bitlik zamanlarında bu altyapıda bu kalıp kullanılıyordu ve o dönemde Windows kodu yazanlar onu bilirler.

3.2 GoF örneği: pencere soyutlaması

Dörtlü Çete'nin örneğinde bir GUI çatısı (framework) hazırlanıyor ve bir pencere soyutlaması gerekiyor. Çatı GNOME, KDE gibi farklı masaüstü ortamlarını desteklemek istiyor. Bu yüzden soyut bir pencere sınıfı olur, her masaüstü ortamı için ondan kalıtımla bir alt sınıf türetilir.

Bu ilk bakışta karmaşık görünmez. Fakat ek bir ihtiyaç doğar: pencereyi ikon hâline getirmek (simge durumuna küçültmek) ya da saatin yanındaki ikonlar gibi sürekli hafızada kalan bir uygulamaya çevirmek. Mevcut pencere kodundan en çok ölçüde yararlanmak istiyoruz. O zaman kalıtım hiyerarşisi karışır: `IconWindow` gelir, ama onun da her ortam için alt sınıfları olmak zorundadır (X Window ikon penceresi, Presentation Manager ikon penceresi, GNOME ikon penceresi, KDE ikon penceresi). Tek bir yeni özellik için, desteklenen aile sayısı kadar alt sınıf kodlamak gerekir.

Bu bizi şu soruya götürür: pencere soyutlamasıyla pencerelerin somut gerçeklemelerini ayrı sınıf hiyerarşilerinde, ayrı ayrı özelleştirebilir miyiz? Hocaya göre bu soruyu sordurtan işaret şudur:

> **Hocanın vurgusu:** Yeni bir işlev için eklenmesi gereken alt sınıf sayısı çok artıyorsa, kalıtım hiyerarşisi doğru kurulmuyor demektir; farklı bir şekilde kurulup kurulamayacağı düşünülmelidir.

Dörtlü Çete'nin önerisi şudur: pencere soyutlamasını bir sınıf hiyerarşisinde, pencere gerçeklemelerini ayrı bir sınıf hiyerarşisinde tasarla. İşlemler soyutlamalar üzerinden çağrılsın, ama asıl işi gerçeklemeler yapsın. İki hiyerarşi birbirinden kopuk kalmamalı, üstte bir sahiplik ilişkisiyle bağlanmalıdır. Kalıp adını bu bağdan, köprüden alır.

Çözümün şeması şöyledir (ekrandan, 00:30:00):

```
Window {DrawText(), DrawRect()} ◇── imp ──> WindowImp {DevDrawText(), DevDrawLine()}
   △                                              △
   ├─ IconWindow {DrawBorder()}                   ├─ XWindowImp  {DevDrawText(), DevDrawLine()}
   └─ TransientWindow {DrawCloseBox()}            └─ PMWindowImp {DevDrawLine(), DevDrawText()}

DrawRect()     : imp->DevDrawLine() dört kez
DrawBorder()   : DrawRect(); DrawText()
DrawCloseBox() : DrawRect()
XWindowImp     : XDrawLine(), XDrawString() çağırır
```

Şemadaki elmas bire-çok ilişki gösterse de ilişki birebir de olabilir. Pencere soyutlamasında dikdörtgen çizmek ve metin yazmak gibi işlevler vardır, ama işler burada yapılmaz. Bir pencere çizmek için dikdörtgen çizmek gerekir; elimizdeki gerçeklemenin ise çizgi çizme metodu

vardır. Dört çizgiden bir dikdörtgen oluşur. Bu yüzden `Window` sınıfının `DrawRect()` metodu, gerçeklemenin `DevDrawLine()` metodunu uygun koordinatlarla dört kez çağırarak dikdörtgeni çizer.

Böylece iki hiyerarşinin alt sınıfları ayrı ayrı kodlanabilir. Normal pencere için bir somut alt sınıf, ikon hâline gelebilen pencere için `IconWindow`, ileride şeffaf pencere gerekirse onun için yeni bir kardeş sınıf eklenir. Kardeşlerle ortak kod üst sınıfta somut metot olarak durabilir. Desteklenecek her pencereleme ortamı (window manager) için de `WindowImp`'in bir alt sınıfı eklenir. Bazı ortamlarda bazı işlevler yoksa ilgili yerlere ilgili metotlar eklenir; ama birçok farklı yere benzer kodlar eklemek gerekmez.

3.3 Kalıbın yapısı ve katılımcıları

Genel yapıda bir `Abstraction` (soyutlama) vardır ve onun farklı gerçeklemeleri olabilir. Gerçeklemeler `Implementor` adlı soyut ata altında toplanır. Soyutlamanın işlemleri, gerçeklemenin ilgili işlemlerini çağırır. Farklı gerçeklemeler `Implementor`'ın alt sınıflarıdır; soyutlamanın farklı özelleşmeleri gerekiyorsa onlar da soyutlamanın alt sınıfları olarak eklenir.

- **Implementor:** uygulamanın iç gerçekleme arayüzünü belirler.
- **ConcreteImplementor:** bu arayüzü gerçekler.
- **Abstraction:** uygulamanın temel işlevlerinin arayüzüdür.
- **RefinedAbstraction:** bu arayüzde özelleşme ya da farklılaşma gerekiyorsa onların kodlandığı sınıftır.

Kalıp şu durumlarda kullanılır: soyutlama ile gerçekleme arasındaki bağın kalıcı olması istenmediğinde (istekçi bir soyutlama üzerinden farklı gerçeklemelerden istediğiyle çalışabilir) ve hem soyutlamaların hem gerçeklemelerin farklı biçimlerde özelleşmesi gerektiğinde.

4. Bridge Örneği: İki Çizim Kitaplığı ile Şekiller

Hoca bu örneği bir kitaptan aldığını, ama kaynağını yazmayı unuttuğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** DP1 ve DP2 çizim kitaplıklarıyla Shape/Drawing örneği büyük olasılıkla Alan Shalloway ve James R. Trott'un "Design Patterns Explained" kitabından gelmektedir. Aynı kitapta Decorator için makbuz (sales ticket) örneği de büyük olasılıkla yer alır.

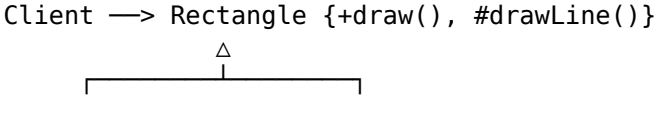
4.1 Problem

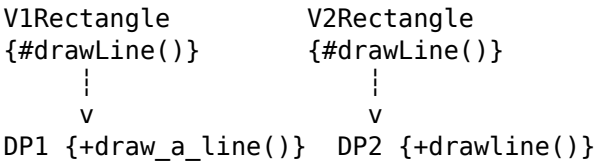
Bir çizim programı üzerinde çalışıyoruz. Önce dikdörtgenlerle başlıyoruz, ama ileride başka şekiller de desteklenecek. İki farklı dikdörtgen türü var. Bir dikdörtgeni çizmek için sol üst ve sağ alt köşelerinin x-y koordinatlarını bilmek yeter. Kenarları çizmek için elimizde iki kod kitaplığı var: DP1 ve DP2. Birinci tür dikdörtgenler DP1'le, ikinci türler DP2'yle çiziliyor. DP1'de koordinatlar `x1, y1, x2, y2` sırasıyla veriliyor; DP2'nin çizgi çizme metodunda ise önce x koordinatları (`x1, x2`), sonra y koordinatları (`y1, y2`) geçiyor. Elimizde bu biçimlerle kaydedilmiş bir sürü çizim dosyası var ve onları atmak değil, kullanmaya ve değiştirmeye devam etmek istiyoruz.

Hoca kendinden bir örnek verdi: ders notlarındaki UML şemalarını çizdiği programın (büyük olasılıkla masaüstünde görünen Violet UML 0.21.1; ekrandan, 01:30:00) birkaç yıl önce yeni sürümü çıkmış, ama kayıt dosyası biçimi değiştiği için yeni sürüme geçemiyor. Eski şemaların hepsini yeniden çizmek mümkün olmadığından eski sürümü kullanmaya devam ediyor. Örnekteki durum da buna benzer: çizimlerdeki bileşenler iki farklı biçimin birinde, karışık olarak kaydedilmiştir.

4.2 İlk çözüm: kalıtım ile iki dikdörtgen

İlk akla gelen, bir kalıtım hiyerarşisi kurmaktır (ekrandan, 00:40:00):





`V1Rectangle`'ın koordinat sistemi DP1'deki çizgi çizme metodunun parametre listesiyle, `V2Rectangle`'ınki ise DP2'ninkiyle uyuşuyor.

Şemada `drawLine` metodu `protected` yapılmış ve UML'de # ile gösterilmiştir. Hoca bunun nedenini tartıştı. Slayttaki açıklama şöyleydi (ekrandan, 00:40:00): `drawLine` metodu nesnenin iç çalışmasıyla ilgilidir, o yüzden public yapılmamalıdır; private metotlar kalıtımla aktarılmaz; protected metotlar hem farklı türden nesnelere görünmez, hem de alt sınıflara kalıtımla aktarılır. İstekçi dört çizgi çizdirmekle uğraşmamalı, yalnız dikdörtgene "kendini çiz" demeli, yani `draw()` metodunu çağırmalıdır. `draw()` kendi içinde `drawLine()` kullanır. Metot alt sınıflarda yeniden tanımlanmak (override) zorunda olduğu için private de olamaz. Hoca, illa protected olması gerekip gerekmediğinin tasarımın tamamına bakarak anlaşılacağını, ama örneğin protected metotlardan söz etmeye bir vesile olduğunu ekledi.

Kodda soyut `Rectangle` sınıfının x ve y koordinatları ve dört çizgi çizen `draw()` metodu vardır. Asıl çizgi çizme işi alt sınıfta gerçekleneceği için `drawLine()` soyuttur. `V1Rectangle` DP1'in `draw_a_line` metodunu, `V2Rectangle` DP2'nin çizgi çizme metodunu çağırarak kendini çizdirir.

4.3 Şekiller çoğaldıkça

Sonra yeni bir gereksinim gelir: daire. Her şekle eş biçimli olarak "kendini çiz", "şu noktaya taşın" gibi komutlar vermek istediğimiz için soyut bir `Shape` sınıfına ihtiyaç olacağı da erkenden görülür. Hiyerarşi şöyle büyür: `Shape` altında soyut `Rectangle` ve soyut `Circle`; `Circle`'ın asıl çizimi yapan `drawCircle()` metodu soyuttur; `V1Circle` DP1'in daire çizme metodunu, `V2Circle` DP2'ninkini uygun parametre sırasıyla çağırır.

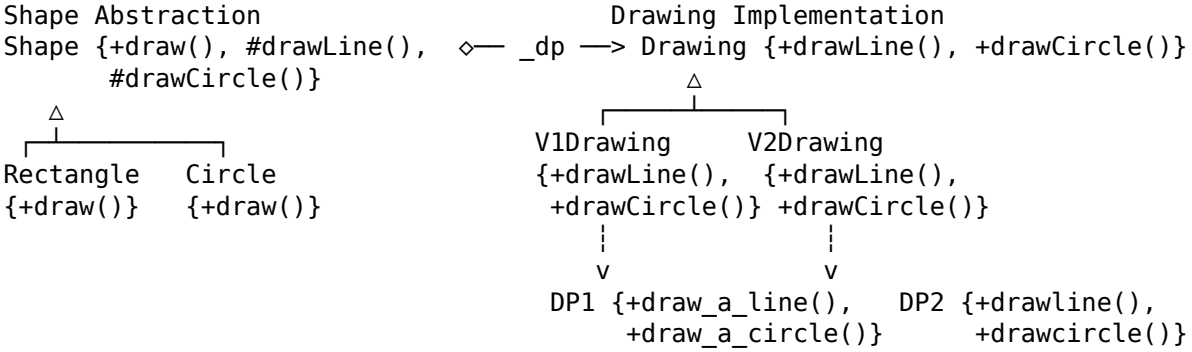
Ardından üçgen gelir, onun da iki ailesi vardır: `Triangle`, `V1Triangle`, `V2Triangle`. Hoca hesabı şöyle yaptı: şu anda iki şekil ve iki aile, yani $2 \times 2 = 4$ somut şekil sınıfı var. Üçgen eklenince sınıf sayısı 9'a çıkacak. Yeni aile de eklenebilir, çizgi ve daire dışında başka çizim işlemleri de gerekebilir. Eklenen her çizim metodu, şekil türü ya da aile sınıf sayısını çarpımsal olarak artırır.

> **Ders sonrası not:** Hocanın söylediği 9 sayısı, `Shape` dışındaki şekil sınıflarını saymaktadır (ara adım eklendi): önce `Rectangle`, `Circle` ve dört somut sınıf olmak üzere 6 sınıf vardır; `Triangle`, `V1Triangle`, `V2Triangle` eklenince 9 olur. `Shape` de sayılırsa 10 sınıf eder.

Sorunun kaynağı şudur: yeni bir şekil gelince çizimler çeşitleniyor, yeni bir çizim gelince şekiller ailelere ayrılıyor. Bu dallanıp budaklanmayı önlemek için ikisini birbirinden ayırırsak, Bridge'de olduğu gibi, sorunu çözebiliriz.

4.4 Bridge kalıbı ile çözüm

Önerilen çözüm iki hiyerarşiden oluşur (ekrandan, 00:47:00):



Şekil ailesinin aile olmasının nedeni farklı çizim kitaplığına dayanmasıdır; asıl farklılaşma orada olur. Çizim işlevlerinin arayüzü soyut `Drawing` sınıfında tanımlanır. `V1Drawing` gerekli

her çizim metodunu DP1'i kullanarak, `V2Drawing` DP2'yi kullanarak kodlar. Şekillerle çizimleri arasına bu köprü kurulunca tasarım geliştirmeye açık hâle gelir.

Üçgen eklemek için yalnız `Shape`'in altına, `Circle`'a kardeş bir `Triangle` sınıfı eklenir. Üçgenin `_dp` üyesine `V1Drawing` verilirse birinci aileden, `V2Drawing` verilirse ikinci aileden üçgen çizilir. Tersinden de düşünülebilir: kayıt dosyasından bir üçgen yüklenir; birinci türden ise bir `V1Drawing`, değilse bir `V2Drawing` nesnesi oluşturulur ve işlem onun üzerinden yapılır. Önceki çözümde üçgen için üç ayrı sınıf gerekirken, bu çözümde tek bir sınıf yeter. İki çözüm arasındaki en önemli fark budur.

4.5 Kod

Slayttaki kod şöyledir (ekrandan, 00:50:00):

```
package bridge.solution2;
public abstract class Shape {
   public abstract void draw( );
   private Drawing _dp;
   public Shape (Drawing dp) { _dp= dp; }
   protected void drawLine(double x1, double y1,
     double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
   protected void drawCircle(double x,double y,
     double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
   private double _x, _y, _r;
   public Circle (Drawing dp,
               double x,double y,double r) {
      super( dp) ;
      _x= x; _y= y; _r= r ;
   }
   public void draw () { drawCircle(_x,_y,_r); }
}
```

Köprüyü kuran, `Drawing` türündeki `_dp` üye alanıdır ve kurucuda verilir. `Shape`'in protected `drawLine` metodu köprünün öbür ucundaki somut gerçeklemenin `drawLine` metodunu çağırır; hangi somut gerçekleme verilmişse (DP1'i ya da DP2'yi kullanan) çizim onun üzerinden yapılır. `Circle` merkez koordinatlarını ve yarıçapı tutar. Kurucusu hem bu değerleri hem köprüyü alır.

Hoca Java'daki bir kuralı hatırlattı: üst sınıfın varsayılan olmayan bir kurucusu varsa, alt sınıfın kurucusu önce `super(...)` ile üst sınıfın o kurucusunu çağırmak zorundadır. Diğer üye alanlar ondan sonra ilklendirilir.

Kodda üye alanların adı alt çizgiyle başlar. Hoca bunun zorunlu olmadığını, C++'tan kalma bir alışkanlık olduğunu söyledi. Bütün koda değil de bir kod parçasına bakıldığında, hangi adın üye alan, hangisinin metot parametresi, hangisinin geçici nesne olduğunu ayırt etmeyi kolaylaştırdığı için birçok kitap bu yolu izler.

`Rectangle` da benzerdir: iki köşesinin koordinatlarını ve çizim gerçeklemesini alır. Çizerken kendi `drawLine` metodunu çağırır; bu metot da aslında çizim gerçeklemesinin `drawLine` metodunu çağırır. `Circle` için de aynısı geçerlidir: üst sınıftan gelen `drawCircle`, köprü üzerinden ilgili gerçeklemenin `drawCircle` metodunu çağırır. Hoca buna "akıllıca bir delegasyon" dedi. `Drawing` tarafında ortak soyut sınıf gerçeklemenin yeteneklerini (çizgi çizmek, daire çizmek) listeler. `V1Drawing` bu görevleri DP1'in metotlarıyla, `V2Drawing` DP2'ninkilerle yerine getirir. DP1 ve DP2'de ise çizgi ve daire çizen çeşitli static metotlar vardır.

4.6 Çıkarılan ders: neyin değiştiğini bul ve sarmala

Hoca bu örnekten çıkan dersi önceki derslerin ilkesine bağladı: "Neyin değiştiğini bul ve onu sarmala." Sarmalamak genellikle uygun bir özelleşme, bir sınıf hiyerarşisi kurmak demektir. Bu problemde şekiller kendi içinde, çizimler kendi içinde değişiyordu. Kendi içinde değişen şeyleri kendi kalıtım hiyerarşisine koymak en doğrusudur. İlk çözümde olduğu gibi bunları çaprazlarsak esnek olmayan bir çözüm elde ederiz: hem bağlaşım (coupling) artar, hem de

yeni bir şekil ya da çizim türü eklemek neredeyse üstel artan sayıda yeni sınıf gerektirir. Neyin değiştiğini bulup sarmalamak bizi Bridge kalıbına götürdü.

Soru: Adapter kalıbında olduğu gibi var olan bir sistemi tasarıma eklemek Bridge ile de mümkün mü? Adapter'ın daha pratik, Bridge'in daha karmaşık olduğunu söylemiştiniz. **Cevap:** Hoca bu paralelliği kurulmasına sevindiğini söyledi. İki kalıp arasında küçük bir benzerlik vardır. Adapter'da, metot arayüzleri bakımından uyumsuz iki nesne birbiriyle konuşturulur. Bridge'de de örneğin birinci türden bir daire çizme ihtiyacı DP1 üzerinden karşılanmak zorundadır ve bunu `V1Drawing` sağlar. Doğrudan çizilmeye kalkılsa önceki esnek olmayan çözümdeki gibi kod yazmak gerekirdi; Bridge ise köprüdeki gerçekleme üzerinden DP2 ailesinden bir çizimi de kolayca yapmayı sağlar. Adapter'da daha çok bir uydurma, Bridge'de ise daha çok bir eşleştirme söz konusudur. Birbirine yakındırlar ama farklıdırlar ve Bridge biraz daha karmaşık bir kalıptır.

5. Composite Kalıbı

5.1 Amaç ve örnekler

Composite, parçalardan, değişik bileşenlerden oluşan anlamına gelir. Kalıbın amacı hiyerarşik parça-bütün ilişkilerini desteklemektir; böylece bir nesneler bütünü ve onun her alt üyesi istekçiye aynı şekilde görünür ve eş biçimli işlenebilir.

Dörtlü Çete'nin örneği yine bir grafik çizim programıdır. Bir resim çeşitli alt resimlerden ve ek olarak çizgilerden, dikdörtgenlerden ve başka şekillerden oluşabilir. PowerPoint'te şekilleri gruplamak da aynı mantıktır: birkaç şekli seçip gruplarsınız, sonra başka şeylerle birlikte yeniden gruplarsınız; bir grup alt gruplar içerebilir.

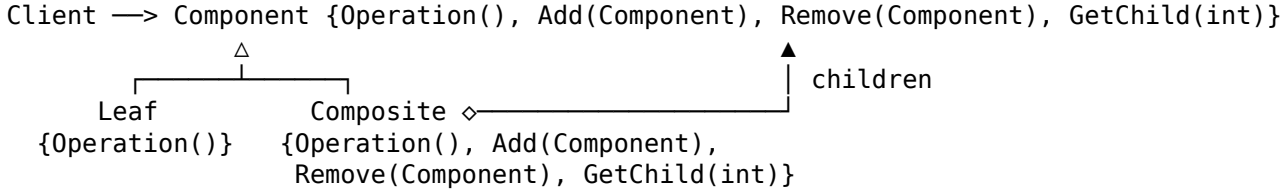
Bunu desteklemek için elimizde çizgi, dikdörtgen, metin gibi tekil (bireysel) bileşenler ve bunların herhangi bir bileşimini içerebilen resimler ya da alt resimler olur. Hepsine eş biçimli erişebilmek için üstte bir soyut arayüz ya da soyut sınıf tanımlanır. Bileşik parçalar çizilebilir, bileşime parça eklenip silinebilir, bileşimin belirli numaralı çocuğuna erişilebilir. Bu işlevlerin hepsi üstteki soyut sınıfta tanımlanır; somut sınıflar kendilerine gerekenleri gerçekler. Bir resim, içerdiği bileşenleri bir veri yapısında (liste, tablo, map) bire-çok parça-bütün ilişkisiyle tutar.

Bir metnin çizme metodu yazıyı yazar; bir çizginin çizme metodu başlangıç ve bitiş noktalarını birleştirir. Bir resmin çizme metodu ise içindeki bütün grafiklerin kendini çizmesini sağlayarak kendini çizer.

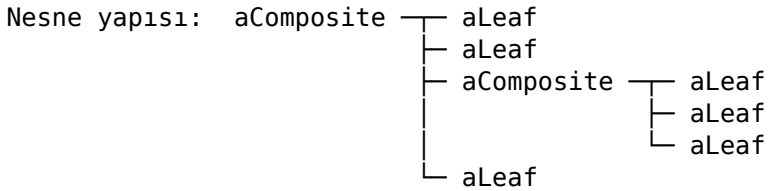
Hoca burada daha sonra açacağı bir rahatsızlığı da önceden söyledi: üstteki grafik sınıfındaki her metodun alttaki her somut sınıfta gerekmemesi.

5.2 Kalıbın yapısı

Slayttaki yapı ve katılımcılar şöyledir (ekrandan, 01:20:00):




```
Composite.Operation(): forall g in children  g.Operation();
```



- **Component:** nesneler topluluğunun katılımcıları için ortak kullanım arayüzü sunar; bir bütünün parçalarına erişim ve düzenleme metotlarını tanımlar.
- **Leaf:** topluluktaki parçaları simgeler; hiyerarşik yapının en altındaki düğümlerdir, çocukları olamaz.
- **Composite:** hiyerarşik yapının ara düğümleridir. Farklı tür ara düğümler varsa bu sınıfın alt sınıfları olarak modellenir.
- **Client:** parça ve toplulukları benzer şekilde kullanabilir. İstekçi bir Visitor olabilir.

Bileşimin bütün üyelerine uygulanacak işlem, grafikte çizim olabilir; bir departman hiyerarşisinde çalışan sayısını bulmak, bir makine ve parçalarında maliyet hesaplamak olabilir. Bu işlem hem bileşimde hem tek tek parçalarda yapılabildiği için üst sınıftadır. Ama yalnız bileşim için anlamlı olan metotlar da, bütün yapıya eş biçimli erişim sağlamak için, mecburen yine üst arayüzde tanımlanır.

Veri yapıları açısından bu bir ağaçtır: en üstteki composite ağacın kökü, en uçtaki düğümler yaprak (leaf), aradakiler ara düğümdür. Yaprağın çocuğu olmaz. Hoca ileride Visitor kalıbının görüleceğini, bu kalıbın bir bütünün parçaları arasında dolaşıp çeşitli işler yapmaya yaradığını söyledi.

Kalıp, nesneler arasındaki hiyerarşik parça-bütün ilişkilerini modellemek ve bir nesneler topluluğuna ve içindeki tekil nesnelere eş biçimli erişim gerektiğinde kullanılır.

5.3 Gerçekleme ayrıntıları

Hoca bu kalıbın önemli ayrıntılarının veri yapılarından geldiğini söyledi.

Alt düğüm üst düğümü bilmeli mi? Her yaprağın hangi composite'in parçası olduğunu bilmesi gerekmiyorsa bu yönde ilişki koymaya gerek yoktur; kalıpta da bu ilişki açıkça gösterilmemiştir. Gerekiyorsa bileşene "ben hangi composite'in içindeyim" bilgisini tutan bir üye alan eklenir. Bu, bir parçaya yapılan işin bütün diğer parçalarda da tekrarlanması gerektiğinde gerekir: yaprağın bir metodu çağrıldığında, üst composite aracılığıyla diğer bütün bileşenlere yayılabilir. Bu bilgi yoksa her işi kök üzerinden yürütmek zorunda kalırız; kök bütün alt parçalarını bildiği için metodu onlara yayabilir, ama bir yaprakla doğrudan konuşamayız.

Bir düğüm nasıl silinir, kim siler? Bir yaprağı silmek sorun değildir. Ama ortadaki bir composite silinecekse içindeki yapraklar da mı silinecek? Bu bir yaklaşımdır. Yaprakların kalması gerekiyorsa yalnız composite silinip alt yapraklar üst composite'e bağlanabilir. Uygulama alanı için hangisi mantıklıysa o kodlanır; senaryoya göre ikisi de gerekiyorsa iki ayrı silme (remove) metodu yazılır. Alt parçaları korumak biraz daha karmaşıktır, ama yapılmayacak bir şey değildir.

Koşullu silme ya da askıya alma. Yapraklardan biri belli bir durumdaysa silme işlemi geçersiz olabilir. Ya da tamamen silmek yerine bir süre askıya almak istenebilir. O zaman bütün bileşenlerde bileşenin etkin olup olmadığını tutan bir üye alan olur ve bir composite metot çağrısını yalnız etkin bileşenlerine gönderir.

5.4 Kalıbın eleştirisi

> **Hocanın vurgusu:** Bu kalıp aslında kalıtım ilişkisinin özüne aykırı bir iş yapıyor.

`Leaf` bir alt sınıftır ve üst sınıftan gelen bütün metotlara sahiptir; ama bunların bazılarının yaprak için anlamı yoktur. Çocuk silmek gibi metotlar yaprak için anlamsızdır, çünkü yaprağın çocuğu olamaz. Bu metotlar üstte soyut tanımlandığı için somut alt sınıfta gövdeleri mutlaka yazılmalıdır. Leaf'i soyut yapamayız, çünkü somut yaprak nesnelerine ihtiyaç vardır. Bu yüzden açılıp kapanan boş gövdeli metotlar yazılır. Bir şey döndürmesi gereken metotlarda, örneğin `GetChild`, gövde `return null` olur. Hoca bunun kendisini ciddi biçimde rahatsız ettiğini, ama yapacak başka bir şey olmadığını söyledi.

Parça-bütün ilişkilerini modellemek için daha akıllıca şeyler yapılabilir, ama bu tasarımı karmaşıklaştırır. Sırf boş gövdeli metotlardan kurtulmak için tasarımı karmaşıklaştırmaya değip değmeyeceği tartışmalıdır; genellikle "o kadar kusur kadı kızında da olur" denip görmezden gelinir. Hoca ayrıca bu kalıpta gerçekleme ayrıntılarının diğer kalıplardan çok daha fazla olduğunu, bunun da bir dezavantaj sayılabileceğini ekledi.

6. Composite Örneği: Makine ve Makine Parçaları Kataloğu

6.1 Problem ve tasarım

Hoca bu örneği "Design Patterns Java Workbook" kitabından biraz değiştirerek hazırladığını söyledi. Bir fabrikayız; makine ve makine parçaları üretiyoruz ve farklı fabrikalarımız var. Ürünlerin hangi fabrikada üretildiği, üretim maliyeti, satış fiyatı gibi bilgilerden raporlar ve kataloglar hazırlamak istiyoruz. Makineler kendi içinde parçalar içerdiği için Composite kalıbı bu yapıya tam uyar.

Tasarımda soyut `ManufacturingItem` sınıfı vardır. Her makinenin ya da parçanın bir ID'si, adı ve üretildiği fabrika vardır. Hoca, fabrikayı ayrı bir sınıf yapmanın daha doğru olacağını, ama problemi karmaşıklaştırmamak için basitleştirdiğini söyledi. Kurucu bu alanların hepsini alır. ID ve ad için getter, fabrika için getter ve setter vardır; setter gereklidir, çünkü bir parçanın üretimi bir fabrikadan diğerine taşınabilir.

Bir parça bir makineye ait olabileceği için üst öğenin ID'si (`parentID`) için getter ve setter metotları da üst sınıfta tanımlıdır. Hoca önce makinelerin alt makineler içerip içermediğini hatırlayamadı ve koda bakarak netleştirdi: örnekte makineler yalnız makine parçası içerir, alt makine içermez. Makinelerin üst öğesi olmadığı için `parentID` alanı `Machine` sınıfına konmamıştır; makineler alt makine içerseydi bu alanın üstteki `ManufacturingItem` sınıfında olması gerekirdi. Bir parça, ait olduğu makineyi nesne olarak tutmaz, yalnız o makinenin ID'sini bilir.

Üst sınıfta ayrıca parçalara erişmek için `getParts()` (bir iterator döndürür), parça eklemek için `addPart()` ve silmek için `removePart()` metotları vardır. Son ikisi boolean döndürür: işlem başarılıysa true, değilse false.

6.2 Machine ve MachinePart kodu

`Machine` sınıfının kodu şöyledir (ekrandan, 01:36:10):

```
public class Machine extends ManufacturingItem {
    private ArrayList<ManufacturingItem> parts;

    public Machine( int machineID, String name, String factory ) {
        super(machineID, name, factory);
        parts = new ArrayList<ManufacturingItem>();
    }

    public int getParentID() { return 0; }
    public void setParentID(int parentID) { }

    public boolean addPart( ManufacturingItem part ) {
        part.setParentID(getMachineID());
        for( ManufacturingItem aPart : parts )
            if( aPart == part )
                return false;
        return parts.add(part);
    }
    public boolean removePart( ManufacturingItem part ) { return parts.remove(part); }
    public Iterator<ManufacturingItem> getParts( ) { return parts.iterator(); }
}
```

Makinenin üst öğesi olmadığı için `getParentID()` 0 döndürür (hoca -1'in de düşünülebileceğini söyledi) ve `setParentID()` boştur. Bunlar, Composite'in eleştirisinde söz edilen boş gövdeli metotlardır. `addPart()` önce parçaya "senin yeni üst öğen benim" der ve kendi ID'sini parçanın üst öğe ID'si yapar. Parça zaten listedeyse ikinci kez eklemez ve false döndürür. Yoksa `ArrayList`'in `add` metodunun sonucunu doğrudan döndürür. `removePart()` da `ArrayList`'in `remove` sonucunu döndürür; olmayan bir parça silinmek istenirse false döner, ayrıca hata mesajı ya da istisna (exception) oluşturmaya gerek görülmemiştir. `getParts()` listenin iterator'ünü döndürür; bu, for-each döngüsünde dolaşılabilir.

`MachinePart` daha basittir. Kendi parçaları olamayacağı için `getParts()` null, `addPart()` ve `removePart()` false döndürür.

6.3 Rapor: bir fabrikada üretilen öğeler

Örnek rapor, belirli bir fabrikada üretilen öğeleri listeler. Fabrikadaki makineler ve parçalar rapora basitçe bir dizi olarak gönderilir. Kod şöyledir (ekrandan, 01:40:00):

```
public class Report {
    public static int findItems( ManufacturingItem items[], String factory ) {
        int nr = 0;
        System.out.println("Items manufactured in " + factory + ":");
        System.out.println("Nr.\tID\tName");
        System.out.println("-----------------------------------");
        for( ManufacturingItem item : items ) {
            if( item==null )                          continue;
            if( item.getFactory().compareToIgnoreCase(factory) == 0 ) { //duplicate code
                nr++;
                System.out.println( nr + ".\t" + item.getMachineID() +
                        "\t" + item.getName() );
            }
            Iterator<ManufacturingItem> subItems = item.getParts();
            if( subItems==null )                        continue;
            while( subItems.hasNext() ) {
                ManufacturingItem part = subItems.next();
                if( part.getFactory().compareToIgnoreCase(factory) == 0 ) { //duplicate code
                    nr++;
                    System.out.println( nr + ".\t" + part.getMachineID() +
                            "\t" + part.getName() );
                    ...
```

Dizi boşluklu olabileceği için null öğeler `continue` ile atlanır. Hoca, bunun yerine `if (item != null)` deyip bütün gövdeyi o bloğun içine koymanın da mümkün olduğunu, ama `continue`'nun "burada bir şey yapmıyorum" fikrini daha iyi vurguladığını söyledi. Yalnız noktalı virgül koymanın aynı işi yapmayacağını da düzeltti: noktalı virgül boş bir komuttur, yürütme alt satıra geçer; `continue` ise döngünün o yinelemesini atlatır.

Fabrikası uyan bir öğe bulunursa sayaç artırılır ve öğenin ID'si ile adı yazdırılır. Bu yetmez, çünkü öğenin alt parçaları olabilir. Öğenin parçaları alınır; parçası yoksa sonraki yinelemeye geçilir, varsa her alt parça için aynı fabrika karşılaştırması ve yazdırma yapılır.

6.4 Ana program ve çalışma sonucu

Ana programda dört elemanlı bir dizi kurulur (ekrandan, 01:50:00). `MachinePart` kurucusunun ilk parametresi parçanın ID'si, ikincisi üst öğenin ID'sidir.

```
ManufacturingItem[ ] items = new ManufacturingItem[4];
items[0] = new MachinePart(99, 127, "Krank şaftı KŞ-...      // Bursa
Machine jenerator = new Machine(135, "Jeneratör JMX-...     // İstanbul
jenerator.addPart( new MachinePart(259, 135, "Flanş ...      // Bursa
jenerator.addPart( new MachinePart(378, 135, "Kasnak ...     // İstanbul
jenerator.addPart( new MachinePart(196, 135, "Rulman ...     // Ankara
items[1] = jenerator;
items[2] = new MachinePart(63, 76, "Distribütör DST-...      // Bursa
items[3] = new MachinePart(82, 19, "Pnömatik Vana VP...      // Ankara
int sayi = Report.findItems(items, "Bursa");
```

Karede satırların sağı kesik olduğu için fabrika adları altyazıdaki anlatımdan alınmıştır. 127 ID'li bir makine listede tanımlanmamıştır; hoca bunun önemli olmadığını, önemli olanın tanımlanan parça ve ilişkilerin hepsinin bulunup raporlanması olduğunu söyledi. Jeneratöre eklenen parçaların üst öğe ID'si olarak 135 yazılmıştır; `addPart` zaten bunu ayarladığı için başka bir sayı da yazılabilirdi, ama takibi kolay ve tutarlı olsun diye doğru ID yazılmıştır.

Bursa için beklenen sonuç üç öğedir: krank şaftı, jeneratörün alt parçası olmasına rağmen flanş ve tek başına bir öğe olan distribütör. Çalışma sonucu şöyledir (ekrandan, 01:50:00):

```
Items manufactured in Bursa:
Nr.     ID      Name
---------------------------------
1.      99      Krank şaftı KŞ-7
2.      259     Flanş FTR-5
3.      63      Distribütör DST-12
        3 parça bulundu.
```

İstanbul'da üretilen jeneratör gibi Bursa'da üretilmeyen bir öğe listelenmemiştir; kod doğru çalışmıştır.

6.5 ReportV2: tekrarlanan kodu metoda çıkarmak

Hoca ardından `ReportV2` sınıfını `Report` ile karşılaştırdı. İlk raporda hem makinenin kendisi hem alt parçaları için aynı üç komut (fabrika adını karşılaştır, sayacı artır, öğeyi yazdır) tekrarlanıyordu; kodda bu yerler `//duplicate code` yorumuyla işaretlenmişti. Aynı komutları birden fazla yerde tekrarlamak yerine onları ayrı bir metotta toplamak en doğrusudur. `ReportV2`'de bu iş `showItem` adlı bir metoda çıkarılmıştır (extract). Metot, gösterilecek öğeyi ve aranan fabrikayı parametre olarak alır; fabrika adı da gerekir, çünkü ortak kısmın olabildiğince geniş alınması için karşılaştırma satırı da metoda taşınmıştır. Böylece `findItems` gövdesi daha sade ve odaklı hâle gelir: öğe null değilse `showItem` çağrılır, alt parçaları varsa her biri `showItem`'a gönderilir. İki rapor da aynı üç öğeyi bulmuştur.

7. Decorator Kalıbı

7.1 Amaç

Decorator, süslemek, dekore etmek demektir. Kalıbın amacı nesnelere çalışma anında, yani dinamik olarak ek sorumluluk atamaktır. Normalde bir nesneye ek sorumluluk atamak için alt sınıf oluşturur ve ona yeni metotlar ekleriz. Fakat bu kodlama zamanında yapılan bir iştir; kalıtım ilişkisi bir kez kurulunca çalışma anında değiştirilemez. Sahiplik ya da parça-bütün gibi ilişkiler kullanılırsa işlevler dinamikleşir ve çalışma anında farklı biçimde ayarlanabilir. Nesneye yeni sorumluluk atama işini de çalışma anında yapmak Decorator kalıbıyla mümkündür.

7.2 Örnek: metin alanına kaydırma çubuğu ve çerçeve

Yine bir GUI çatısı hazırlıyoruz. Çok satırlı metin alanlarımız var. Bazen bunlara çerçeve, bazen yatay, dikey ya da iki yönlü kaydırma çubuğu eklemek istiyoruz. Her işi bir kez yapmak istiyoruz. Bütün varyantlar (çıplak, çerçeveli, tek kaydırmalı, çift kaydırmalı, çerçeveli tek kaydırmalı, çerçeveli çift kaydırmalı) için ayrı sınıf yazmak hem çok sayıda sınıf hem de bir sürü tekrarlanan kod demektir.

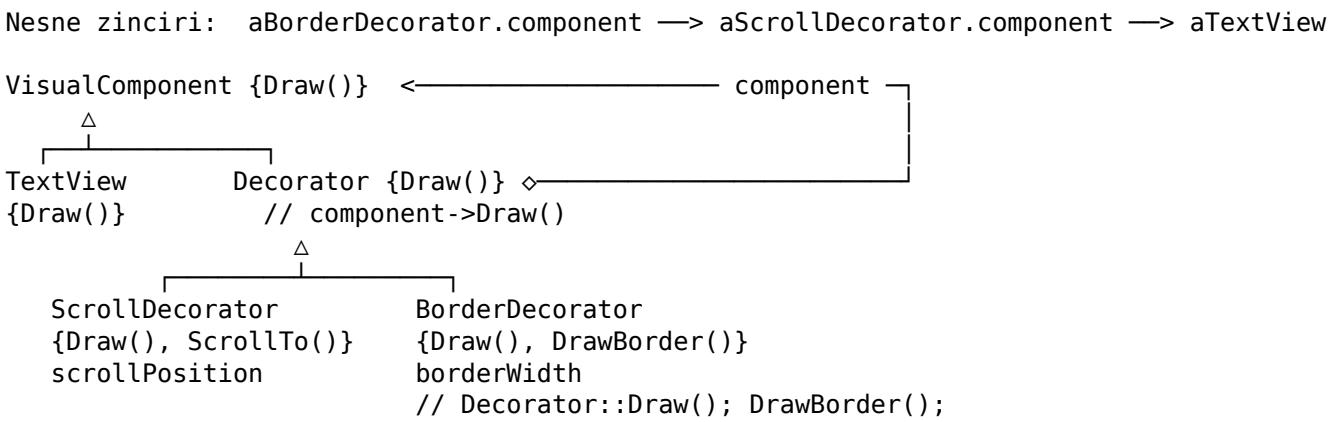
Birinci çözüm: kalıtım. `TextArea`'dan kalıtımla `ScrollTextArea`, `BorderTextArea` ve `ScrollBorderTextArea` kodlanır. Hoca yatay ve dikey kaydırmayı ayırmadığını, ayırırsa sınıf sayısının daha da artacağını belirtti. Kaydırma işi hem `ScrollTextArea`'da hem `ScrollBorderTextArea`'da, çerçeve işi hem `BorderTextArea`'da hem `ScrollBorderTextArea`'da kodlanmak zorundadır. Ortak işleri üst sınıfa taşımaya çalışırsak alt sınıfların anlamı kalmaz ve devasa bir üst sınıf oluşur. Paneller için de kaydırma ve çerçeve gerekirse daha fazla sınıf ve daha fazla tekrarlanan kod gerekir.

İkinci çözüm: çoklu kalıtım (ekrandan, 02:10:00). `ScrollTextArea` ve `BorderTextArea` `TextArea`'dan türer; `ScrollBorderTextArea` ikisinden birden türetilir. Çoklu kalıtım (multiple inheritance) ek sorunlar doğurur. `TextArea`'da soyut bir metot olsun; iki alt sınıf bunu ayrı ayrı yeniden tanımlar. En alttaki sınıf bu metodu yeniden tanımlamazsa hangi üst sınıftakini alacaktır? Bu bir belirsizliktir. C++ bu durumda o metotların en alt sınıfta mutlaka yeniden tanımlanması kuralını koyar. Yine de önemli bir karmaşıklık sayıldığı için modern nesneye yönelik dillerde, örneğin Java ve C#'ta, sınıflar arası çoklu kalıtım desteklenmez; programcılar arayüzlerle farklı tasarımlara yönlendirilir. Slayttaki zayıf yönler şunlardır: çoklu kalıtım her

dilde desteklenmez; bu çözümdeki çoklu kalıtım biçimi “diamond problem” sorununun bir örneğidir; `TextArea`’ya ek olarak bir `Panel` sınıfına kaydırma ve çerçeve yeteneği kazandırmak için üç sınıf daha gerekir; üstelik bu durumda kaydırma ve çerçeve yeteneklerinin tek yerde gerçeklenmesi avantajı ortadan kalkar.

7.3 Kalıbın önerdiği çözüm

Slayttaki önerinin başlığı şudur: “Sınıfları değil de, nesneleri birbirleri ile ilişkilendirelim” (ekrandan, 02:13:20).



Metin alanı, kaydırma bileşeni ve çerçeve bileşeni ayrı sınıflardan ayrı nesnelerdir. Normal bir `TextView` gerekiyorsa sorun yoktur. Kaydırma gerekiyorsa bir `ScrollDecorator` nesnesine `TextView` verilir ve işler bu dekoratör üzerinden yürütülür: kaydırmayla ilgili işleri kendisi yapar, metinle ilgili işleri ilişkili olduğu metin nesnesine iletir. Ek olarak çerçeve de gerekiyorsa `BorderDecorator`’a bu iki nesneden oluşan zincir verilir; çerçeve işlerini kendisi yapar, kaydırma işlerini `ScrollDecorator`’a, metin işlerini de onun üzerinden `TextView`’a iletir. Kaydırma çubuklu bir panel gerekiyorsa bir panel nesnesi başka bir `ScrollDecorator`’a verilir; yalnız çerçeveli panel gerekiyorsa bir `BorderDecorator`’a. Böylece önceki sorunların hepsi ortadan kalkar.

Tasarımda ortak, soyut bir `VisualComponent` sınıfı ve onun soyut çizme metodu vardır. `TextView`, panel gibi bileşenler bundan türeyen kardeş sınıflardır. Bileşenlere ek sorumluluk kazandıran `Decorator` sınıfı da soyuttur ama somut bir çizme metodu vardır. Hoca şemadaki bileşen ilişkisinin bir dizi ya da bağlı liste gibi çizildiğini, aslında bire-bir sahiplik ilişkisinin daha doğru olduğunu söyledi: bir dekoratör tek bir başka dekoratör ya da bileşenle ilişkilidir, yani burada elmas kullanmak yeni standartlara göre pek doğru değildir. Kazandırılacak her yetenek `Decorator`’ın bir alt sınıfında kodlanır.

`BorderDecorator`’ın çizme metodu önce sarmaladığı bileşenin çizilmesini sağlar, sonra kendi `DrawBorder` metodunu çağırarak çerçeveyi çizer. `ScrollDecorator`’ın çizme metodu önce `TextView`’un çizilmesini sağlar, sonra kaydırma çubuklarını ekler; çubuklara tıklanıp `ScrollTo` çağrılınca, metnin ilgili satırından itibaren yazacak şekilde `TextView`’un metnini alıp yazar. Başka yetenekler gerekirse onlar da kardeş sınıflar olarak eklenir.

7.4 Genel yapı ve kullanım

Genel kalıpta bütün bileşenlerin ana işleminin (operation) imzasını içeren bir `Component` vardır. Bu bir arayüz olabilir, soyut sınıf olmak zorunda değildir. Fakat `Decorator` soyut sınıf olmak zorundadır, çünkü en azından `operation` metodu orada somut olarak kodlanır. `Component`’in diğer alt sınıfları somut bileşenlerdir (ConcreteComponent). `Decorator`’ın altında farklı somut dekoratörler (ConcreteDecorator) bulunur; her biri yeni bir davranış (added behavior) tanımlar ve bu davranışın normal işlemden önce ya da sonra yürütülmesini sağlar.

Kalıbın kullanılabileceği anlar slayttaki gibidir (ekrandan, 02:20:00):

- Tek tek nesnelere dinamik olarak (çalışma anında) yeni sorumluluklar eklenmek istendiğinde.
- Tek tek nesnelere şeffaf olarak (diğerlerini etkilemeden) yeni sorumluluklar eklenmek istendiğinde.

- Bazı sorumlulukların iptal edilebileceği veya askıya alınabileceği durumlarda. Örneğin dekoratör, sarmaladığı nesnenin işlemini belirli bir koşula göre çağırmayabilir, ortam olgunlaşınca çağırmaya başlayabilir.
- Kalıtımın uygun olmadığı anlarda; örneğin sınıf kodu mevcut değilse ya da yeni eklemeler üstel artan sınıf sayısına neden olacaksa. Hoca bunun çok tercih edilen bir kullanım olmadığını, ama özelleştirmek istediğimiz sınıfın kaynak kodu yoksa mecburen onu bir dekoratörün içine gömüp eklemeleri dekoratör üzerinden yapacağımızı söyledi.
- Java'nın stream tabanlı G/Ç kütüphanesi bu kalıbı kullanır.

Hoca kalıbın Swing çatısında, yani GUI işlemlerinde de çok kullanıldığını söyledi. Java'da dosya işlemleri stream'ler üzerinden yürür. Temelde iki tür stream vardır: dosyaya bayt yazan output stream ve dosyadan bayt okuyan input stream. Blok okumak, sıkıştırılmış dosya oluşturmak ya da açmak, dosyayı şifreleyerek yazmak ya da şifresini çözerek okumak gibi yetenekler, ilgili stream'leri uç uca ekleyip dekore ederek zincirleme kullanılır.

> **Ek bilgi (derste anlatılmadı):** Örneğin `new BufferedInputStream(new GZIPInputStream(new FileInputStream("veri.gz")))` zincirinde dosyadan okunan baytlar önce sıkıştırmadan açılır, sonra tamponlanır. Her sınıf bir `InputStream` alır ve kendisi de bir `InputStream`'dir; bu, Decorator yapısının ta kendisidir.

7.5 Kalıbın zayıf yönleri

Slayttaki zayıf yön şudur (ekrandan, 02:20:00): yeni sorumluluk ancak eski sorumluluğun tamamen iptal edilmesiyle ya da eski sorumluluktan önce ve/veya sonra yeni komutlar eklenmesiyle eklenebilir. Somut dekoratör, işlemini çağırırken ya önce ek davranışı yürütüp sonra sarmaladığı işlemi çağırır, ya önce işlemi çağırıp sonra ek davranışı yürütür, ya da işlemi hiç çağırmadan yalnız ek davranışı yürütür. Bu üç varyasyon dışında ince ayar yoktur: dekore edilen işlemin adımlarının içine girip bir kısmını değiştirmek mümkün değildir.

İkinci dikkat noktası nesne zincirinin kopabilmesidir. Örnekte bir `TextView`'un artık kaydırmaya ihtiyacı kalmadıysa `ScrollDecorator`'ı aradan çıkarmak, onun üstündeki bütün zinciri kırar; çerçeve ile metin arasındaki ilişki de bozulur. Zincirin kırılması istenmiyorsa bileşenin kendisini dekore eden varlığı bilmesi (o yöne de bir ok), ekleme metodunun yanında dikkatle kodlanmış bir silme metodu gerekir. İlişki kurmak kolay, değiştirmek zordur. Hoca, ilişkileri değiştirmeye çalışmaktansa mevcut nesneyi alıp yepyeni bir zincir kurmanın daha pratik olabileceğini söyledi.

8. Decorator Örneği: Makbuza Üst Bilgi ve Alt Bilgi Eklemek

Hoca bu örneği iyi bir kitaptan aldığını, ama kaynağı işaretlemeyi unuttuğunu söyledi.

8.1 Problemin evrimi

Bir e-ticaret uygulaması için makbuz basıyoruz. Bir satış emri geldiğinde, `TaskController` adlı mevcut uygulamaya satış emirlerinin makbuzlarını bastırma yeteneği kazandırmak istiyoruz. Uygulamada zaten bir `SalesOrder` sınıfı var; ona `prtTicket()` metodu eklenir. Makbuz basmak basit bir iş olmayacağı (neyin hangi düzende gösterileceği, ara toplamlar, farklı biçimler) için bu iş tek bir metoda değil, `SalesTicket` adlı yeni bir sınıfa verilir. `SalesOrder`, işlemden sonra ilişkili olduğu `SalesTicket`'ın `prtTicket()` metodunu çağırır.

Sonra yeni bir istek gelir: makbuza üst bilgi (header) ve alt bilgi (footer) de basılmalı. Bunlar da yeterince uzun ve karmaşık olacağı için iki yeni sınıf oluşturulur: `prtHeader()` metodu olan `Header` ve `prtFooter()` metodu olan `Footer`. `SalesTicket.prtTicket()` önce üst bilgiyi, sonra makbuz gövdesini, sonra alt bilgiyi basar.

Ardından üst bilgi ya da alt bilginin her zaman istenmeyeceği anlaşılır. Çözüm 1, kullanım ilişkisi ve if sınamalarıdır (ekrandan, 02:30:00): `prtTicket(hasHeader:boolean, hasFooter:boolean)`. Slayttaki not şöyledir: "if want header info, call Header's prtHeader method; call SalesTicket's prtTicket method; if want footer info, call Footer's prtFooter method."

Sonra farklı üst bilgi ve alt bilgi türleri istenir. Hoca slayt üzerinde çizerek gösterdi: `Header` ve `Footer` soyut yapılır, altlarına `Header1`, `Header2`, `Footer1`, `Footer2` gibi kardeş sınıflar eklenir. Fakat `prtTicket` metodunun parametre listesi bu sefer aşırı uzar. Boolean yerine bir

header enum'ı ve bir footer enum'ı kullanılabilir; ama o zaman `if (hasHeader)` yerine `switch (headerType) { case NONE: ... case 1: ... case 2: ... }` gibi uzun bir switch-case kurulur.

> **Hocanın vurgusu:** Kendinizi uzun bir switch-case kurarken görürseniz bu çok önemli bir alarm işaretidir: daha doğru bir kalıtım ilişkisi tasarlayıp çok biçimliliği (polymorphism) daha doğru kullanmanız gerektiği anlamına gelir.

Hoca önce şunu önerdi: `SalesTicket` bir `Header` ve bir `Footer` (soyut sınıf ya da arayüz) ile ilişkili olsun; hangi somut tür gerekiyorsa o verilsin, gerekmiyorsa null verilsin. `prtTicket()` içinde, null değilse header'ın `prtHeader()` metodu çağrılır, sonra makbuz gövdesi basılır, sonra null değilse footer'ın `prtFooter()` metodu çağrılır. Hoca bunu söyledikten sonra kendini düzeltti: bu fena bir çözüm değil, ama Decorator değil; aslında iki kez Strategy kalıbı kullanılmış oldu (header için bir, footer için bir).

Son istek bu tasarımı da yetersiz bırakır: aynı anda birden fazla header ve birden fazla footer basılması gerekebilir. Örneğin bir makbuzda Header1, Header2, gövde ve Footer2; başka bir makbuzda yalnız Header1, gövde, Footer1 ve Footer2 gerekebilir. O zaman makbuzu hangi header ve footer'larla ilişkilendirmek gerekiyorsa onlarla dekore etmek, yani Decorator kalıbını kurmak gerekir. Bu üçüncü ve önerilen çözümdür.

8.2 Önerilen çözümün yapısı

Kalıbın gerektirdiği gibi soyut bir `Component` tanımlanır ve işlemi (`prtTicket`) onda yer alır. Asıl dekore edilecek varlık (`SalesTicket`) onun alt sınıfıdır. Bütün dekoratörlerin ortak üst sınıfı olarak bir `TicketDecorator` oluşturulur; o da bir bileşenle ilişkilidir. Farklı header ve footer'lar bu dekoratörün alt sınıflarıdır. Hoca, header ve footer'ların kendi yazma metotlarının private yapılabileceğini, çünkü bir makbuzla ilişkili olmayan bir header'a "yaz" demenin anlamsız olduğunu, ama bunun şart olmadığını söyledi.

Bir header'ın `prtTicket()` metodu önce kendi üst bilgisini yazar, sonra ilişkili olduğu nesneye "sen de işin geri kalanını yap" der. O nesne başka bir header ise o da kendi bilgisini yazıp sıradakine devreder. Bir footer ise önce ilişkili olduğu nesnenin işini yapmasını ister, sonra kendi dipnotunu yazar. Zincirin en sonunda bir `SalesTicket` bulunmak zorundadır; o da makbuz gövdesini basar.

Hoca şüpheci yaklaşmayı önerdi: bu kodla gerçekten footer'lar sona, header'lar başa eklenecek mi? Bunu görmenin yolu kodu yazıp çalıştırmaktır.

8.3 Kod

Slayttaki kod şöyledir (ekrandan, 02:40:00 ve 02:44:20):

```
package decorator.example;
public abstract class Component {
    abstract public void prtTicket();
}
public class SalesTicket extends Component {
    public void prtTicket() {
        // place sales ticket printing code here
        System.out.println(getClass().getName());
    }
}
public abstract class TicketDecorator extends Component {
    private Component myTrailer;
    public TicketDecorator (Component myComponent) {
        myTrailer= myComponent;
    }
    public void callTrailer () {
        if (myTrailer != null) myTrailer.prtTicket();
    }
}
public class Footer1 extends TicketDecorator {
    public Footer1 (Component myComponent) {
```

```
        super(myComponent);
    }
    public void prtTicket() {
        super.callTrailer();
        // place printing footer 1 code here
        System.out.println(getClass().getName());
    }
}
```

`Footer2` de yalnız sınıf adı değişerek aynıdır. `Header1` ve `Header2`'de sıra terstir: önce sınıf adı yazılır, sonra `callTrailer()` çağrılır. `Component` hiçbir metot gövdesi içermediği için arayüz olarak da tanımlanabilirdi. `TicketDecorator`'daki null denetimi, gerçekten ilişkili bir varlık yoksa boşuna `NullPointerException` almamak içindir. Hoca, `callTrailer` alt sınıfta yeniden tanımlanmadığı için başındaki `super.` önekinin yazılmasa da olacağını belirtti. Gerçek bir uygulamada header'ların yazdıkları farklı olurdu; karmaşıklık katmamak için hepsi yalnız kendi sınıf adını yazar: header başlık olarak, footer dipnot olarak, `SalesTicket` ortada.

getClass().getName() ve Java Reflection. Bir nesnenin sınıf adına ulaşmak için `getClass().getName()` çağrılır. Java'da bütün sınıfların ortak bir "kozmik" üst sınıfı vardır: `Object`. `public abstract class Component` yazdığımızda bu aslında `extends Object` demektir; yazılsa da yazılmasa da sanal makine her sınıfın üst sınıfı olarak `Object`'i kullanır. Nesnelerin dinamik bellek yönetimi gibi, sanal makinenin her nesne için ortak yapması gereken işlevler buraya kodlanmıştır. Ayrıca büyük C ile yazılan `Class` sınıfı vardır; her nesnenin bu türden meta bilgisine erişilebilir: hangi metotları ve üye alanları var, adı ne, paketi ne. `getName()` nesnenin tam nitelikli sınıf adını (paket adı nokta sınıf adı) verir. Merak edenler için hoca "Java Reflection" anahtar sözcüğünü verdi. Bundan da öteye giden bayt kodu değiştirme araçları vardır; örneğin BCEL (Byte Code Engineering Library). Bunlar, çalışma anında metot gövdelerini değiştirmeye kadar giden ve bazı tasarım kalıplarının yaptıklarından çok daha fazlasını yapabilen açık kaynak araçlardır. Ama öğrenme maliyeti vardır ve projenin ayrılmaz parçası olurlar; böyle egzotik yeteneklere gerçekten ihtiyaç varsa kullanılabilirler.

8.4 İstekçi ve Factory

Hoca burada önemli bir soru sordu: istekçi (client) sınıf, belirli bir header-footer birleşimiyle çalışmayı nasıl öğrenecek? Bir an Header1 ve Footer1, başka bir an başka bir birleşim gerekebilir. GUI tabanlı bir uygulamada ayarlardan seçtirilebilirdi. Komut satırında ise ya istekçinin içine bir şeyler yazılır ya da bu iş için bir fabrika kullanılır: istekçiye bir fabrika nesnesi verilir ve fabrika istenen türde header ve footer bileşenleriyle yapılandırılır. Örnekte bu tercih edilmiştir. İstekçi bileşeni fabrikadan alır ve onun `prtTicket()` metodunu çağırır.

Fabrikanın kodu şöyledir (ekrandan, 02:50:00):

```
package decorator.example;
public class Factory {
    public Component getComponent () {
        Component myComponent;
        myComponent= new SalesTicket();
        myComponent= new Footer1(myComponent);
        myComponent= new Header1(myComponent);
        return myComponent;
        //VEYA: Aşağıdaki kod aynı etkiyi yapar
        //return(new Header1(new Footer1(new SalesTicket())));
    }
}
```

Slayttaki çalışma sonucu `HEADER1`, `Receipt`, `FOOTER1`'dir. Hoca nesne zincirinin bellekte nasıl kurulduğunu adım adım anlattı. Önce `Component` türünden bir nesne işaretçisi tanımlanır ve bir `SalesTicket` oluşturulup ona atanır. Sonra yeni bir `Footer1` oluşturulurken bu işaretçi parametre olarak verilir ve ortaya çıkan nesne yine aynı işaretçiye atanır. Önceki `SalesTicket` nesnesi silinmez, bellekteki yerinde durur; `Footer1` onu gösterir, işaretçi ise artık yeni `Footer1`'i gösterir. `Header1` için de aynısı yapılır. Böylece tek işaretçi yeniden kullanılır, diğer iki nesne bellekte yaşamaya devam eder. Aynı zincir tek satırda da kurulabilir: bir `SalesTicket`

oluşturulur oluşturulmaz `Footer1` kurucusuna, o da oluşur oluşmaz `Header1` kurucusuna verilir ve sonuç hiçbir işaretçiye atanmadan döndürülür. API'lerle uğraşıp nesneleri zincirleme kuranlar bu yazıma aşinadır.

Header ve footer'ın birbirine göre sırası önemli değildir: önce footer sonra header ya da önce header sonra footer ile sarmalamak aynı çıktıyı verir, çünkü header önce kendini yazıp sonra devreder, footer önce devredip sonra kendini yazar. Hoca bunu Eclipse'te iki sırada da çalıştırıp gösterdi; çıktı her seferinde Header1, SalesTicket, Footer1 oldu.

8.5 Birden fazla header ve footer: sıra önemli

Kalıbın avantajı birden fazla header ve footer verebilmektir. Tek satırlık zincire `Header2` eklendiğinde (`new Footer1(new Header1(new Header2(new SalesTicket())))`) çıktı Header1, Header2, SalesTicket, Footer1 oldu (ekrandan, 02:54:40).

Uzun yazıma dönülüp önce `Header1`, sonra `Header2` ile sarmalandığında ise çıktıda önce Header2, sonra Header1 basıldı. Aynı türden iki dekoratör arasında sıra önemlidir. Header'lar arasında Header1'in önce gelmesi isteniyorsa, Header1'in zincire önce değil sonra verilmesi gerekir. Footer'larda da aynı mantık işler: makbuza önce sarılan footer (gövdeye yapışık olan) önce basılır.

Hocanın son denemesi (ekrandan, 03:00:00) şuydu:

```
myComponent = new SalesTicket();
myComponent = new Header2(myComponent);
myComponent = new Header1(myComponent);
myComponent = new Footer2(myComponent);
myComponent = new Footer1(myComponent);
...
return(new Header1( new Header2( new Footer2( new Footer1( new SalesTicket()))));
```

Önce zincirde `Footer2` gövdeye yapışıkken (`new Header1(new Header2(new Footer1(new Footer2(new SalesTicket()))))`) çıktı Header1, Header2, SalesTicket, Footer2, Footer1 oldu. Footer1, Footer2 sırasını elde etmek için zincirli yazımda footer'ları ters çevirmek gerekti; son satırdaki zincirle çıktı Header1, Header2, SalesTicket, Footer1, Footer2 oldu. Kural şudur: gövdeye en yakın sarılan dekoratör gövdeye en yakın basılır. Header'larda dıştaki en üstte, footer'larda dıştaki en altta yer alır. Zincirleme yazımda bu sıra, kodun yazılış sırasıyla daha kolay görülür.

9. Örnek Sorular

Soru 1 (derste sözlü, 01:44:54): Makine kataloğu örneğinde dizide krank şaftı (99, Bursa), jeneratör makinesi (135, İstanbul; parçaları flanş 259 Bursa, kasnak 378 İstanbul, rulman 196 Ankara), distribütör (63, Bursa) ve pnömatik vana (82, Ankara) vardır. `Report.findItems(items, "Bursa")` hangi öğeleri bulur?

> **Çözüm (derste verildi):** Üç öğe bulunur: 99 Krank şaftı KŞ-7, 259 Flanş FTR-5 ve 63 Distribütör DST-12 (ekrandan, 01:50:00). Flanş jeneratörün alt parçası olduğu hâlde bulunur, çünkü rapor her öğenin alt parçalarını da dolaşır. Jeneratör İstanbul'da üretildiği için listelenmez.

Soru 2 (derste sözlü, 00:44:57 ve 00:45:21): Bridge örneğinin ilk (kalıtımlı) çözümünde dikdörtgen ve daire için iki çizim kitaplığı vardır. Üçgen eklenirse kaç sınıf gerekir? Bridge çözümünde kaç yeni sınıf gerekir?

> **Çözüm (derste verildi):** İki şekil ve iki aile için $2 \times 2 = 4$ somut şekil sınıfı vardır. Hoca üçgen eklenince sınıf sayısının 9'a çıkacağını söyledi: `Rectangle`, `Circle` ve dört somut sınıf (6) ile `Triangle`, `V1Triangle`, `V2Triangle` (3). İlk çözümde üçgen için 3 yeni sınıf gerekir. Bridge çözümünde yalnız `Shape`'in altına bir `Triangle` sınıfı eklenir; hangi aileden çizileceği `_dp` üyesine verilen `V1Drawing` ya da `V2Drawing` nesnesiyle belirlenir.

Soru 3 (derste sözlü, 02:45:00): Decorator örneğinde istekçi (client) sınıf, belirli bir header-footer birleşimiyle ve sırasıyla çalışmayı nasıl öğrenir?

Çözüm (derste verildi): GUI tabanlı bir uygulamada ayarlardan seçtirilebilir. Komut satırında istekçiye bir fabrika nesnesi verilir; fabrikanın `getComponent()` metodu istenen dekoratör zincirini kurup döndürür, istekçi de yalnız `prtTicket()` çağırır.

Soru 4 (ekrandan, 02:54:40 ve 03:00:00): Aşağıdaki zincirler için çıktı sırası nedir?

- a) `new Footer1(new Header1(new Header2(new SalesTicket())))`
- b) `new Header1(new Header2(new Footer1(new Footer2(new SalesTicket()))))`
- c) `new Header1(new Header2(new Footer2(new Footer1(new SalesTicket()))))`

Çözüm (derste verildi; sonuçlar python ile aynı mantık canlandırılarak doğrulandı): a) Header1, Header2, SalesTicket, Footer1. b) Header1, Header2, SalesTicket, Footer2, Footer1. c) Header1, Header2, SalesTicket, Footer1, Footer2. Header önce kendini yazıp devrettiği, footer önce devredip sonra kendini yazdığı için dıştaki header en üstte, dıştaki footer en altta basılır.

Soru 5 (derste sözlü, 00:56:24): Adapter ile Bridge arasında bir benzerlik var mı? Bridge de var olan bir sistemi tasarıma eklemek için kullanılabilir mi?

Çözüm (derste verildi): İki kalıp arasında küçük bir paralellik vardır: ikisi de iki tarafı birbirine bağlar. Adapter'da var olan, metot imzaları uyumsuz iki nesne birbirine uydurulur. Bridge'de ise bir soyutlama, köprü üzerinden farklı gerçeklemelerle eşleştirilir. Bridge daha karmaşık bir kalıptır.

Dersin Toparlaması

Bu derste yaratımsal kalıplar Singleton ile kapandı ve yapısal kalıplara geçildi. Singleton, tek nesne oluşturma sorumluluğunu sınıfın kendisine verir. Adapter, imzaları uyumsuz bir sınıfı istekçinin beklediği arayüze uydurur. Bridge, soyutlama ile gerçeklemeyi iki ayrı hiyerarşiye ayırıp bir sahiplik ilişkisiyle bağlar. Böylece şekil ve çizim kitaplığı gibi iki bağımsız değişim ekseninin çaprazlanmasıyla doğan sınıf patlaması önlenir. Bu, "neyin değiştiğini bul ve onu sarmala" ilkesinin doğrudan uygulamasıdır. Composite, parça-bütün hiyerarşilerine eş biçimli erişim sağlar, ama yaprakların anlamsız metotlar taşıması gibi bir bedeli vardır. Makine kataloğu örneğinde rapor kodu ayrıca tekrarlanan kodun bir metoda çıkarılmasıyla iyileştirildi.

Decorator ise nesnelere çalışma anında sorumluluk ekler; sınıfları değil nesneleri birbirine bağlar. Makbuz örneği tasarımın adım adım evrimini gösterdi: if sınamaları, enum ve switch-case, iki Strategy kalıbı ve sonunda birden fazla header ve footer için Decorator. Zincirin kuruluş sırası çıktının sırasını belirler. Hoca, ileride Visitor kalıbının Composite yapısı üzerinde dolaşmak için, State ve Strategy kalıplarının da gerçeklemelerinin Singleton olabileceği yerler olarak görüleceğini belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.